



CSE411 Real-Time and Embedded Systems Design

FreeRTOS Passing Structures & Precise Timing

Lab Objectives

By the end of this lab, we will be able to:

- Pass **structures** to tasks
- Understand the difference between `vTaskDelay()` and `vTaskDelayUntil()`
- Create tasks with **precise periodic timing**
- Visualize **timing drift** in the Logic Analyzer

Part 1: Passing Structures to Tasks

In Lab-03, we passed a **single string** to tasks:

```
xTaskCreate(vTaskPrint, "Task 1", 240, (void *)pcString, 1, NULL);
```

Problem: What if we need to pass **multiple parameters**?

- Task name/message
- Delay period
- Task ID
- Any other configuration...

Solution: Pack everything into a **structure**!

Defining a Task Parameter Structure

```
/* Structure to hold task parameters */  
typedef struct {  
    const char *pcMessage;      /* Message to print */  
    TickType_t xDelayPeriod;    /* Delay in ticks */  
} TaskParams_t;
```

Now each task can have **different behavior** based on its parameters!

Creating Task Parameters

```
/* Create parameter structures (MUST be static or global!) */  
TaskParams_t xTask1Params = {  
    .pcMessage = "Task 1 running\n",  
    .xDelayPeriod = pdMS_TO_TICKS(500)    /* 500ms */  
};  
  
TaskParams_t xTask2Params = {  
    .pcMessage = "Task 2 running\n",  
    .xDelayPeriod = pdMS_TO_TICKS(1000)    /* 1000ms */  
};
```


One Function, Multiple Behaviors

```
void vTaskFunction(void *pvParameters)
{
    /* Cast parameter back to our structure type */
    TaskParams_t *pxParams = (TaskParams_t *)pvParameters;

    for (;;)
    {
        vPrintString(pxParams->pcMessage);
        vTaskDelay(pxParams->xDelayPeriod);
    }
}
```

Creating Tasks with Structures

```
int main(void)
{
    xTaskCreate(vTaskFunction, "Task1", 240,
               (void *)&xTask1Params, 1, NULL);

    xTaskCreate(vTaskFunction, "Task2", 240,
               (void *)&xTask2Params, 1, NULL);

    vTaskStartScheduler();
    for (;;)
}
```

Benefits:

- One function for multiple tasks
- Easy to add more tasks
- Each task fully configurable

Part 2: The Timing Problem

Question: How do you run a task every 100ms?

Attempt 1: Use `vTaskDelay()`

```
void vTask(void *pvParameters)
{
    for (;;)
    {
        DoWork(); /* Takes 20ms */
        vTaskDelay(pdMS_TO_TICKS(100));
    }
}
```

What's the actual period?

vTaskDelay Causes Drift!

Timeline with vTaskDelay():

```
|-- 20ms work --|---- 100ms delay ----|-- 20ms work --|---- 100ms delay ----|  
|<----- 120ms period ----->|<----- 120ms period ----->|
```

Problem: Period = Work Time + Delay Time = **120ms**, not 100ms!

Over 10 cycles:

- **Expected:** $10 \times 100\text{ms} = 1000\text{ms}$
- **Actual:** $10 \times 120\text{ms} = 1200\text{ms}$
- **Drift:** 200ms behind!

vTaskDelayUntil - The Solution

```
void vTask(void *pvParameters)
{
    TickType_t xLastWakeTime = xTaskGetTickCount();

    for (;;)
    {
        DoWork(); /* Takes 20ms */
        vTaskDelayUntil(&xLastWakeTime, pdMS_TO_TICKS(100));
    }
}
```

Key Difference:

- vTaskDelay(): "Sleep for 100ms **from now**"
- vTaskDelayUntil(): "Wake me at **next 100ms mark**"

vTaskDelayUntil Timeline

Timeline with vTaskDelayUntil():

```
|-- 20ms work --|-- 80ms delay --|-- 20ms work --|-- 80ms delay --|  
|<----- 100ms period ----->|<----- 100ms period ----->|
```

The delay **automatically adjusts** for work time!

- Work took 20ms → Delay is 80ms → Period is **exactly 100ms**
- Work took 30ms → Delay is 70ms → Period is **exactly 100ms**

When to Use Which?

Use `vTaskDelay()` when:

- Simple one-time delays
- Timing precision not critical
- Response to events

Use `vTaskDelayUntil()` when:

- Periodic tasks (sensors, control loops)
- Precise timing required
- Consistent frequency matters